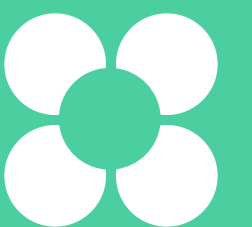


Переменные, константы, типы данных

Александр Голубь
Тимлид команды разработки



Александр Голубь

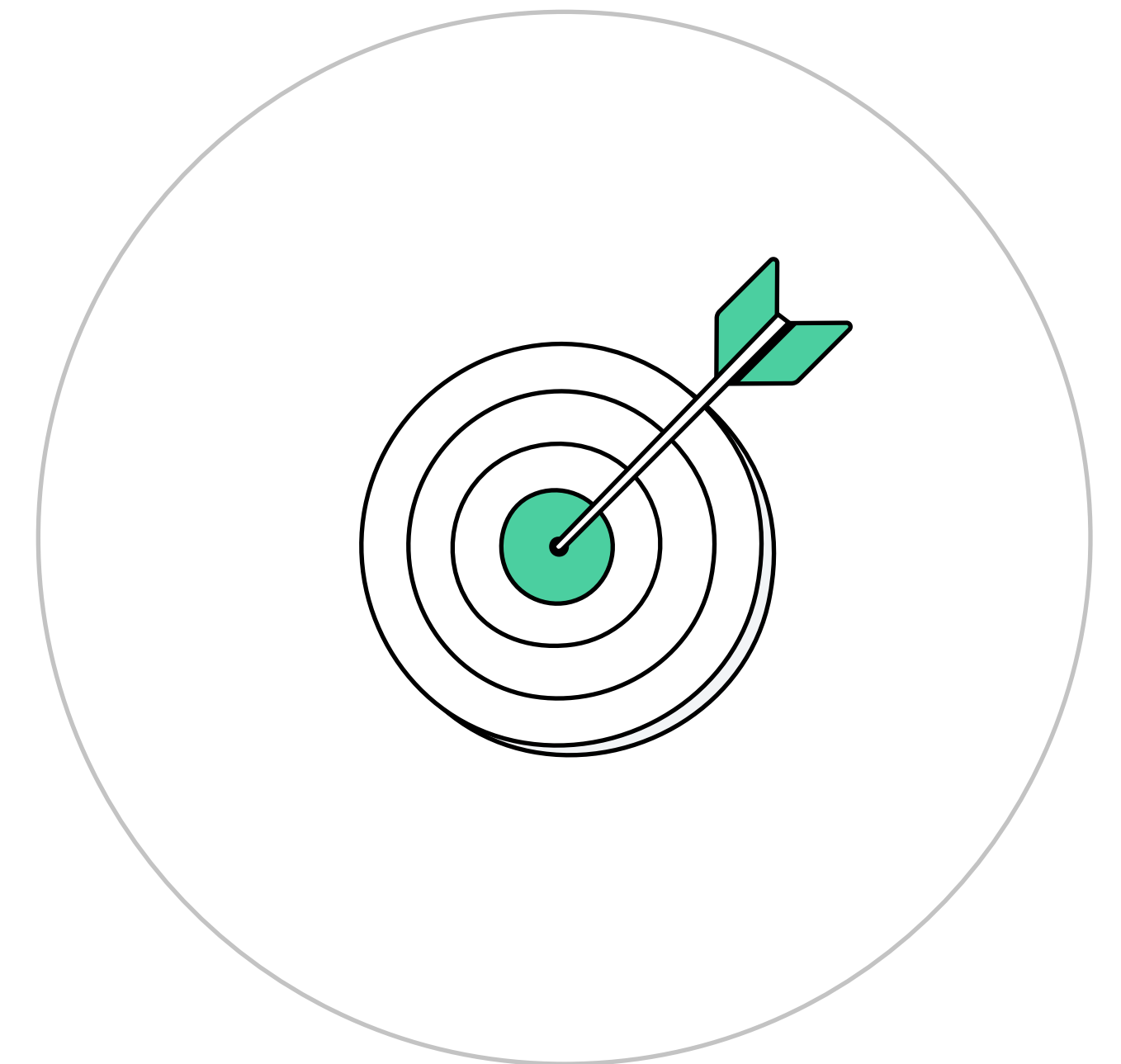
О спикере:

- Тимлид команды разработки ГК Sima-land
- Пишу на Go с 2018 года
- Веду команду из 15 разработчиков



Цели занятия

- Узнать, как использовать переменные и константы для управления данными в программе
- Научиться объявлять, инициализировать и манипулировать переменными различных типов
- Понять, как работать с разными типами данных, преобразованиями и указателями
- Разобраться, что такое область видимости переменных и констант и как их использовать в различных контекстах



План занятия

- 1 Переменные и константы
- 2 Примитивные типы данных
- 3 Составные типы данных
- 4 Прочие типы данных

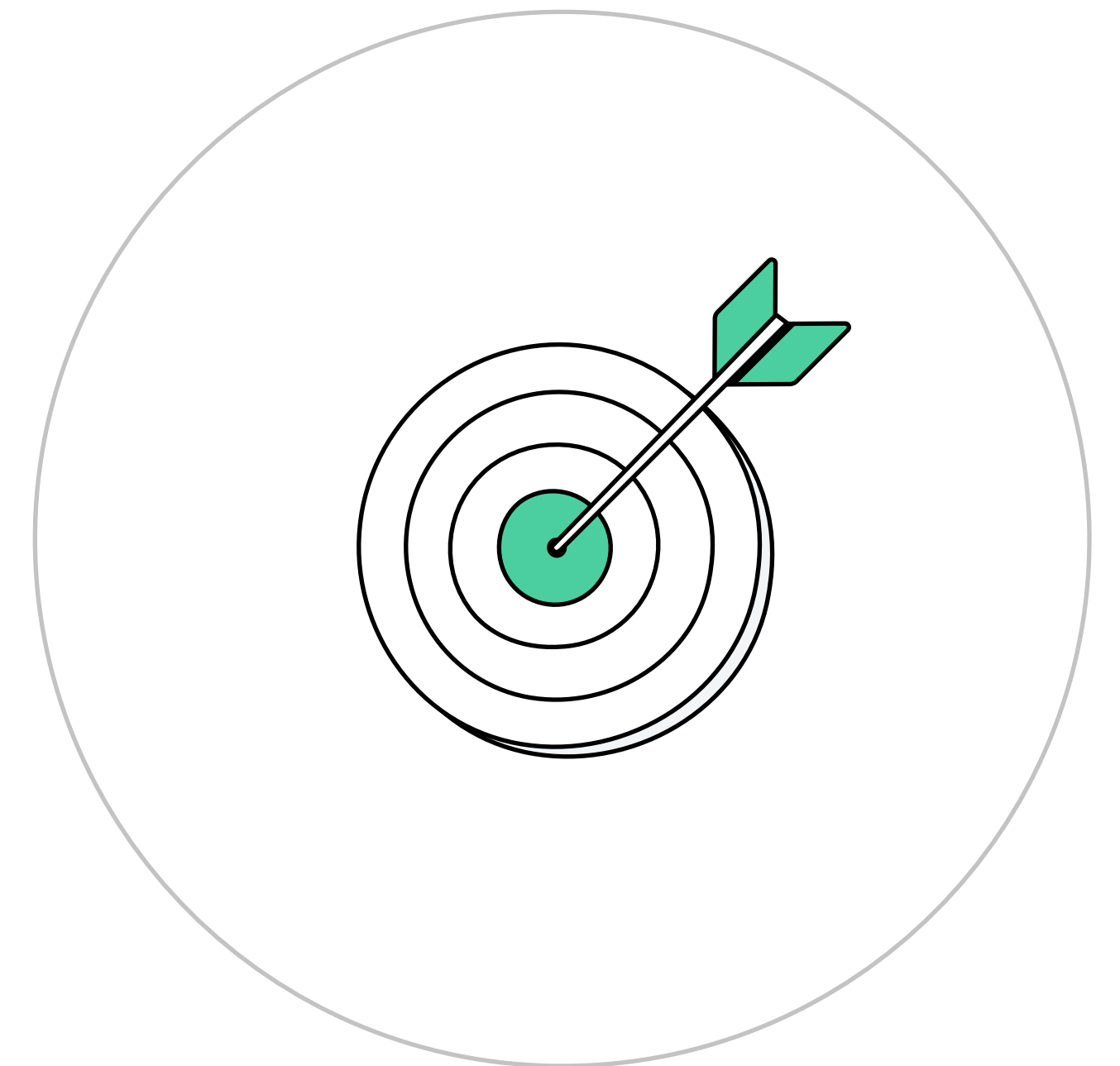


Переменные и константы



Цели видео

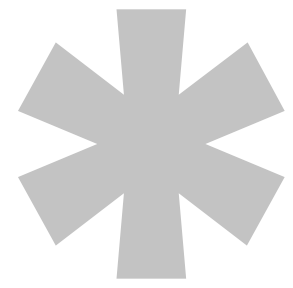
Познакомиться с переменными и константами в языке Go



Переменная

Представьте, что память — это куча картонных коробок. Чтобы понять, что находится внутри каждой коробки, не заглядывая внутрь, на них можно наклеить этикетки с названиями вроде «Стекло» или «Одежда». Эти названия и являются переменными





Переменная

В информатике переменная – это поименованное место в памяти компьютера.

По именам переменных проще всего искать их значения.

Основное свойство переменной — её значение можно изменить в любой момент времени

Объявление переменных

Декларация с использованием `var` — это полная форма декларации переменной

```
var age int = 30  
var name string = "Alice"  
var isStudent bool = true
```

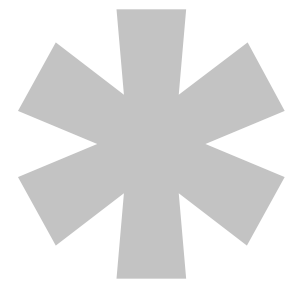
- `var` — ключевое слово, объявление переменной
- `age`, `name`, `isStudent` — имена переменных, по которым компьютер ищет значения в памяти
- `int`, `string`, `bool` — типы переменных (целочисленная, строка, булево)
- после «`=`» указывается значение переменной. Его можно задать позже

Объявление переменных

В декларации с использованием короткого объявления (`:=`) не нужно указывать тип явно. Компилятор сам определит тип на основании присвоенного значения

```
age := 30  
name := "Alice"  
isStudent := true  
temperature := 36.6
```

Этот способ работает только внутри функций и удобен для быстрого объявления переменных. Но он может снизить читаемость, если тип не очевиден из присвоенного значения



Константа

Константа в Go — это именованное значение, которое остаётся неизменным на протяжении всей работы программы.

Значение константы должно быть известно на этапе компиляции.

Константы позволяют повысить читаемость кода, сделать его более предсказуемым и избежать «магических чисел» или строк

Объявление константы

Объявление констант похоже на полную форму объявления переменной. Только вместо ключевого слова **var** используется **const**

```
const PI float64 = 3.14159  
const AppName string = "My Application"
```

Константы, в отличие от переменных, после объявления становятся недоступными для изменения.

Главный плюс константы — чрезвычайно быстрый доступ к её значению

Область видимости переменных и констант

Область видимости определяет, где в коде переменная доступна для использования.

Если переменная «живёт» в области видимости, вы можете её использовать. Если нет — программа не сможет её найти и выдаст ошибку

Область видимости переменных и констант

- **Глобальная область видимости:** Переменные, объявленные вне функций, видны во всем коде программы. Их можно использовать где угодно
- **Локальная область видимости (функция):** Переменные, объявленные внутри функции, видны только внутри этой функции
- **Блочная область видимости:** Переменные, объявленные внутри блоков кода (например, внутри `if` или `for`), видны только внутри этого блока. Это помогает избежать случайных конфликтов имён

```
var globalVar int = 10 // Глобальная переменная
func myFunction() {
    localVar := 20 // Локальная переменная
    println(globalVar, localVar) // 10 20
}
func main() {
    myFunction()
    println(localVar) // Ошибка: localVar не видна здесь!

    for i := 0; i < 2; i++ {
        innerBlockVar := i * 2
        println("Внутри цикла for:", innerBlockVar)
    }
    println(innerBlockVar) // Ошибка: innerBlockVar не видна здесь!
}
```

Область видимости переменных и констант

Итоги видео

- Узнали, как объявляются переменные и константы в языке Go
- Поняли, что такое область видимости переменных и констант в языке Go

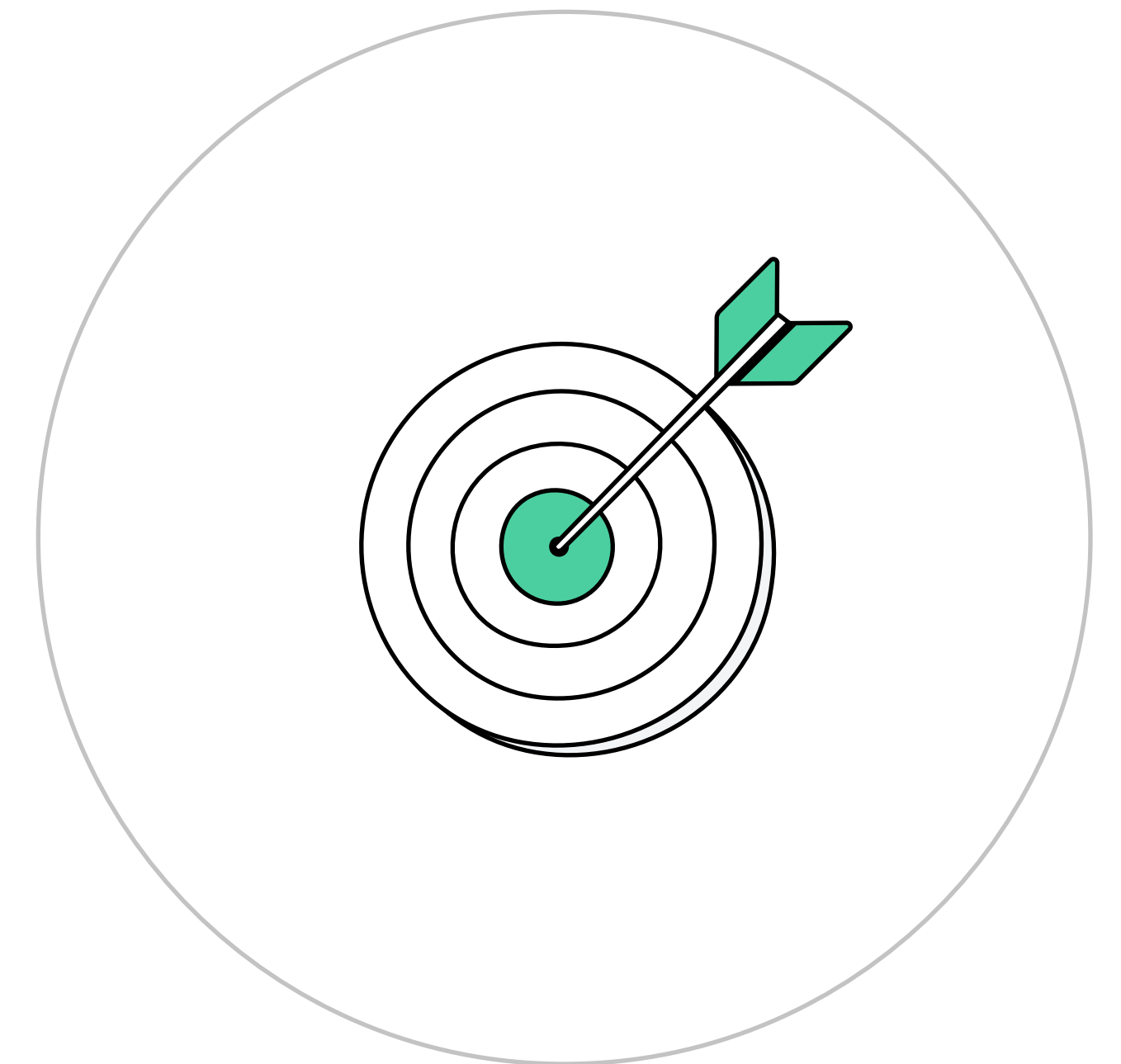


Примитивные типы данных



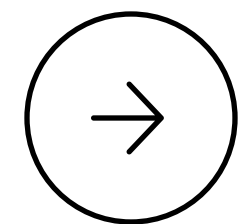
Цели видео

Изучить строковые, числовые и логические
типы данных в языке Go



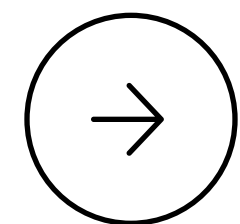
Примитивные типы данных

Примитивные типы данных в языке Go подразделяются на:

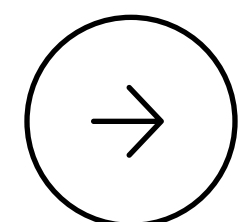


Числовые

- целочисленные – int8/uint8, int16/uint16, int32/uint32, int64/uint64
- вещественные – float32, float64, complex64, complex128



Строковые – string



Логические – bool

Числовые типы данных

Целочисленные типы

Целочисленные типы данных предназначены для хранения и выполнения арифметических операций с числами без дробной части (например, -3, 0, 5, 100)

```
var age int = 30 // Возраст  
absoluteZero := -273 // Локальная переменная  
for i := 0; i < 2; i++ {} // Счетчик в цикле тоже целочисленный
```


Числовые типы

Целочисленные типы подразделяются на:

- **знаковые (signed)** — представляют как положительные, так и отрицательные числа
- **беззнаковые (unsigned)** — представляют только неотрицательные числа, имеют префикс «u» перед названием (**uint**, **uint16** и так далее)

```
var age uint = 30 // Возраст не может быть отрицательным
```

Если в процессе выполнения программы к такой беззнаковой переменной вы попытаетесь задать отрицательное значение, то получите ошибку

Размеры целочисленных типов

Типы данных *	Минимальное значение	Максимальное значение
int8	-128	127
int16	-32,768	32,767
int32	-2,147,483,648	2,147,483,647
int64	-9,223,372,036,854,775,808	9,223,372,036,854,775,807
uint8	0	255
uint16	0	65,535
uint32	0	4,294,967,295
uint64	0	18,446,744,073,709,551,615

* Есть еще два типа — int и uint. Их фактические значения зависят от архитектуры системы (32-битная или 64-битная), а минимальные и максимальные значения равны или uint32/int32, или uint64/int64

Числовые типы

Вещественные типы

Вещественные типы данных в Go подразделяются на:

- **Типы с плавающей точкой (`float32` и `float64`)** — нужны для представления чисел, не являющихся целыми, например, 3.14, 2.718 или -0.5. Применяются для вычислений, связанных с графикой, физикой, финансами и не только
- **Комплексные числа (`complex64` и `complex128`)** используются для представления чисел, состоящих из действительной и мнимой части. Применяются, к примеру, в операциях связанных с обработкой сигналов или квантовой механикой

Числовые типы

Вещественные типы

Выбор между `float32` и `float64` зависит от требуемой точности:

- **`float32`** — обеспечивает около 6–7 значащих цифр
- **`float64`** — около 15–16 значащих цифр

```
var a float32 = 1.123456789
var b float64 = 1.123456789
fmt.Println(a) // 1.1234568 – округлит до 7 знаков после запятой
fmt.Println(b) // 1.123456789
```

Преобразование числовых типов

В Go можно преобразовывать тип значения к другому типу, причём оно должно быть всегда явное!

Для этого используется синтаксис **Тип(значение)**

```
var a int = 42
var b float64 = float64(a) // 42.0
var c uint = uint(b) // 42
f := 3.9
i := int(f) // i = 3 — дробная часть отбрасывается
```

Арифметические операции с числовыми типами

Go поддерживает стандартные арифметические операции:

```
a := 10
b := 3
fmt.Println(a + b) // 13
fmt.Println(a - b) // 7
fmt.Println(a * b) // 30
fmt.Println(a / b) // 3 (целочисленное деление)
fmt.Println(a % b) // 1 (остаток от деления)
```

Смешивать типы можно, только преобразовав числа к одному типу

Арифметические операции с числовыми типами

Go поддерживает сокращённые формы увеличения и уменьшения на 1:

```
x := 5  
x++ // x = 6  
x-- // x = 5
```

Для арифметики можно использовать сокращённую запись:

```
x := 10  
x += 5 // x = x + 5  
x /= 2 // x = x / 2
```

Приоритет операций

Приоритет арифметических операций в Go такой же, как и в математике:

- Сначала *, /, %
- Потом +, -
- Скобки меняют порядок

```
result := 2 + 3 * 4 // 14
```

```
result := (2 + 3) * 4 // 20
```


Строковый тип

Тип string

В Go тип **string** — это последовательность байтов, кодированных в UTF-8

```
var name string = "Василий"  
profession := "Сварщик"
```

Важно! Тип string является неизменяемым типом данных. Если вы к одной строке добавляете символы, то по факту будет создана новая строка

Операции со строками

```
// Пример 1
```

```
profName := "Сварщик" + "Василий" // Конкатенация строк
```

```
println(profName) // Сварщик Василий
```

```
// Пример 2
```

```
var name string = "Василий"
```

```
println(len(name)) // Вычисление длины строки в байтах, результат = 14
```

В Go строки представлены в виде последовательностей байтов, а не символов.

Каждый символ может занимать от 1 до 4 байтов. Например, символы кириллицы занимают 2 байта, а китайские иероглифы — 4 байта

Логический тип

Булев тип

Может принимать только два значения:
true (истина) или false (ложь)

```
var isStudent bool = true  
isClosed := false
```


Логический тип

- **Использование:** используется для представления условий, флагов и логических операций
- **Операторы:** поддерживает логические операторы: && (AND), || (OR), ! (NOT)
- **Преобразование:** можно преобразовывать другие типы данных в bool (например, ненулевое значение - true, нулевое значение - false)
- **Полезен для:** условных операторов (if, else), циклов (for) и логических выражений

Итоги видео

- Узнали, для чего используются строковые, числовые и логические типы данных в языке Go
- Выяснили, как они объявляются в коде

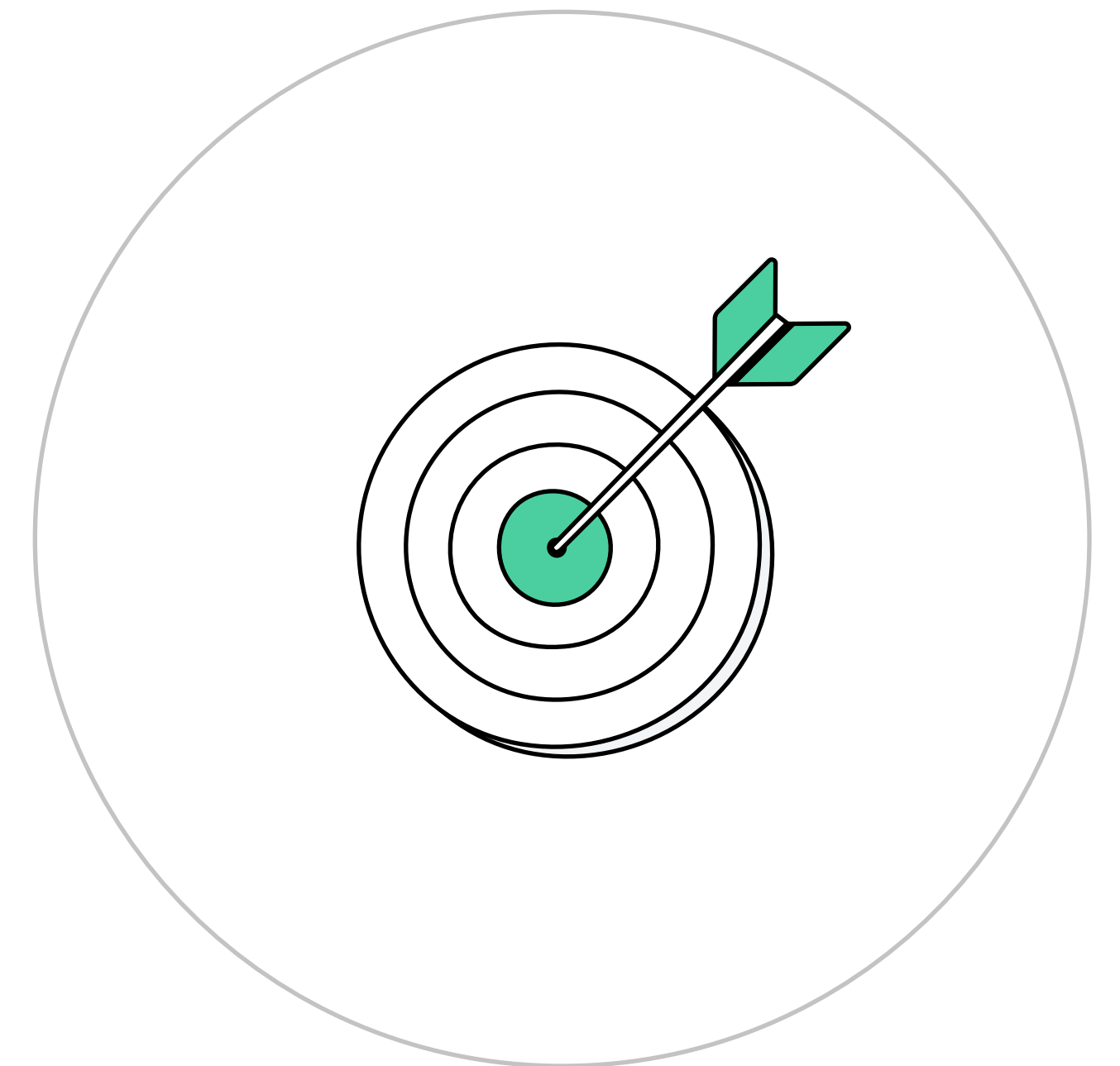


Составные ТИПЫ ДАННЫХ



Цели видео

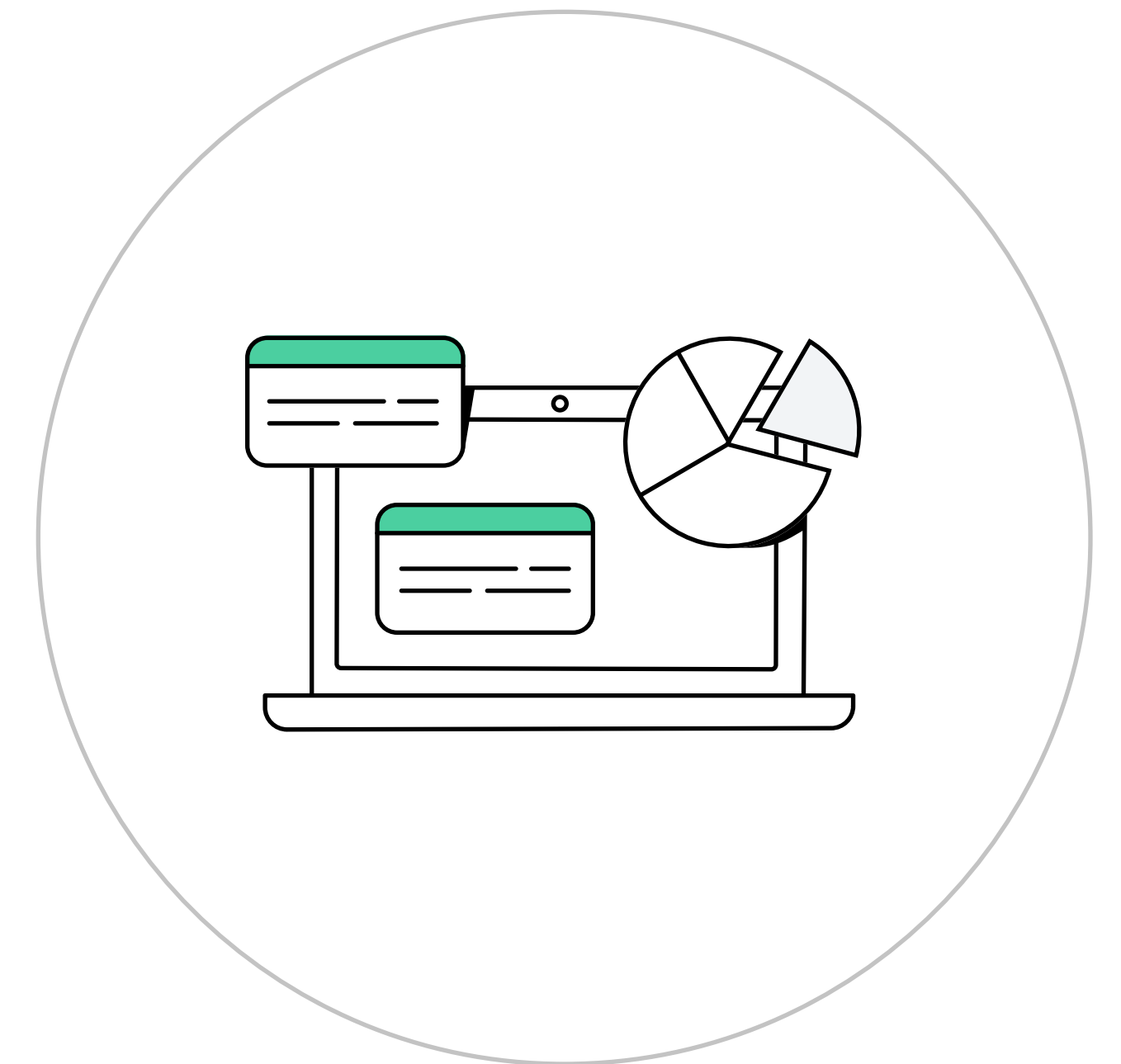
Познакомиться с составными типами данных
в языке Go и изучить основные принципы
работы с ними



Составные типы данных

К составным типам данных в языке Go относятся:

- Массив — array
- Срез (слайс) — slice
- Карта — map
- Структура — struct



Массив

Массив (array) — ЭТО КОЛЛЕКЦИЯ ЭЛЕМЕНТОВ ОДНОГО ТИПА ФИКСИРОВАННОЙ ДЛИНЫ

```
// Пример 1:
```

```
var numbers [3]int // Массив из 3 целых чисел
```

```
// Пример 2:
```

```
words := [...]string{ "Мама", "мыла", "раму" }
```

```
println(words[1]) // мыла
```

```
Words[2] = "окна"
```

```
println(words) // Мама мыла окна
```

Массив

- Размер массива фиксирован при объявлении и не может быть изменён
- Элементы массива доступны по индексу, начиная с 0
- При объявлении без инициализации (как в первом примере), элементы массива автоматически инициализируются нулевым значением для соответствующего типа (0 для `int`, " " для `string`, `false` для `bool`, и т.д.)

Срез (slice)

Срезы (слайсы) — это динамические массивы в Go. Они похожи на массивы, но их размер может изменяться во время выполнения программы

```
// Пример 1:
```

```
var numbers []int // Слайс из целых чисел
```

```
// Пример 2:
```

```
words := []string{ "Мама", "мыла", "раму" } // слайс из строк
```

```
println(words[1]) // мыла
```

```
words[2] = "окна"
```

```
words = append(words, "и", "пол")
```

```
println(words) // Мама мыла окна и пол
```


Срез (slice)

- Срезы не хранят данные сами по себе, а содержат указатель на базовый массив, длину и ёмкость
- Функция **append()** добавляет элементы в конец среза. Если ёмкости базового массива недостаточно, создаётся новый исходный массив большего размера

Длина и ёмкость среза

- **Длина (Length)** — количество элементов, которые слайс содержит в данный момент. Доступно по индексу от 0 до `len(slice) - 1`
- **Ёмкость (Capacity)** — наибольший объём элементов, который может содержать слайс без изменения базового массива. Когда количество элементов превышает ёмкость, Go выделяет новый, больший слайс и копирует в него данные
- При добавлении элементов с помощью **`append()`** длина слайса увеличивается
- **`make()`** — создает слайс с указанным типом. Первый аргумент задает тип слайса, второй - длину слайса, третий — ёмкость

Длина и ёмкость среза

Пример:

```
mySlice := make([]int, 0, 3) // Слайс с длиной 0 и ёмкостью 3
fmt.Println("Длина:", len(mySlice)) // Вывод: Длина: 0
fmt.Println("Ёмкость:", cap(mySlice)) // Вывод: Емкость: 3

mySlice = append(mySlice, 1, 2, 3) // Добавляем элементы в слайс

fmt.Println("Длина:", len(mySlice)) // Вывод: Длина: 3
fmt.Println("Ёмкость:", cap(mySlice)) // Вывод: Ёмкость: 3

// Добавляем еще один элемент, превышающий ёмкость
mySlice = append(mySlice, 4)
fmt.Println("Длина:", len(mySlice)) // Вывод: Длина: 4
fmt.Println("Ёмкость:", cap(mySlice)) // Вывод: Ёмкость: 6
```


Карты

map (или ассоциативный массив) в Go — это коллекция пар «ключ-значение». По сути, это словарь, где каждый ключ сопоставляется с определённым значением

```
// map, где ключ - строка, а значение - целое число
age := map[string]int{
    "Alice": 30,
    "Bob": 25,
    "Charlie": 35,
}
```

Карты

Особенности:

- Неупорядоченность: порядок элементов в map не гарантируется
- Уникальность ключей: каждый ключ в map должен быть уникальным. Если вы попытаетесь добавить дубликат ключа, старое значение будет перезаписано
- Разные типы данных: ключи и значения могут быть разных типов

Карты

Примеры

```
// Декларация переменной без инициализации объекта
var ages map[string]int // ages = nil
ages["Alice"] = 30 // приведёт к вызову panic программы
```

```
// Инициализация
```

```
// В момент объявления
ages := map[string]int{
    "Alice": 30,
    "Bob":   25,
}
```

```
ages := make(map[string]int, 0) // создает пустую карту
ages["Alice"] = 30 // заполняем данными позже
ages["Bob"] = 25
```

Структуры

Структуры (structs) в Go позволяют объединять несколько переменных разных типов в один логический блок.

Они помогают организовать данные, связанные между собой, и делают код более читаемым и поддерживаемым

Структуры

Примеры

```
type Person struct {  
    Name string  
    Age  int  
}  
  
// Вариант создания переменной 1:  
var person1 Person // Создание экземпляра структуры Person  
person1.Name = "Алиса" // Через точку можно записывать информацию в поле  
person1.Age = 30  
  
println("Имя", person1.Name) // И читать из поля тоже  
  
// Вариант создания переменной 2:  
person2 := Person{  
    Name: "Иван",  
    Age: 25,  
} // Создание с использованием литерала структуры
```


Итоги видео

- Познакомились с массивами, срезами, картами и структурами в языке Go
- Узнали, как использовать эти типы данных

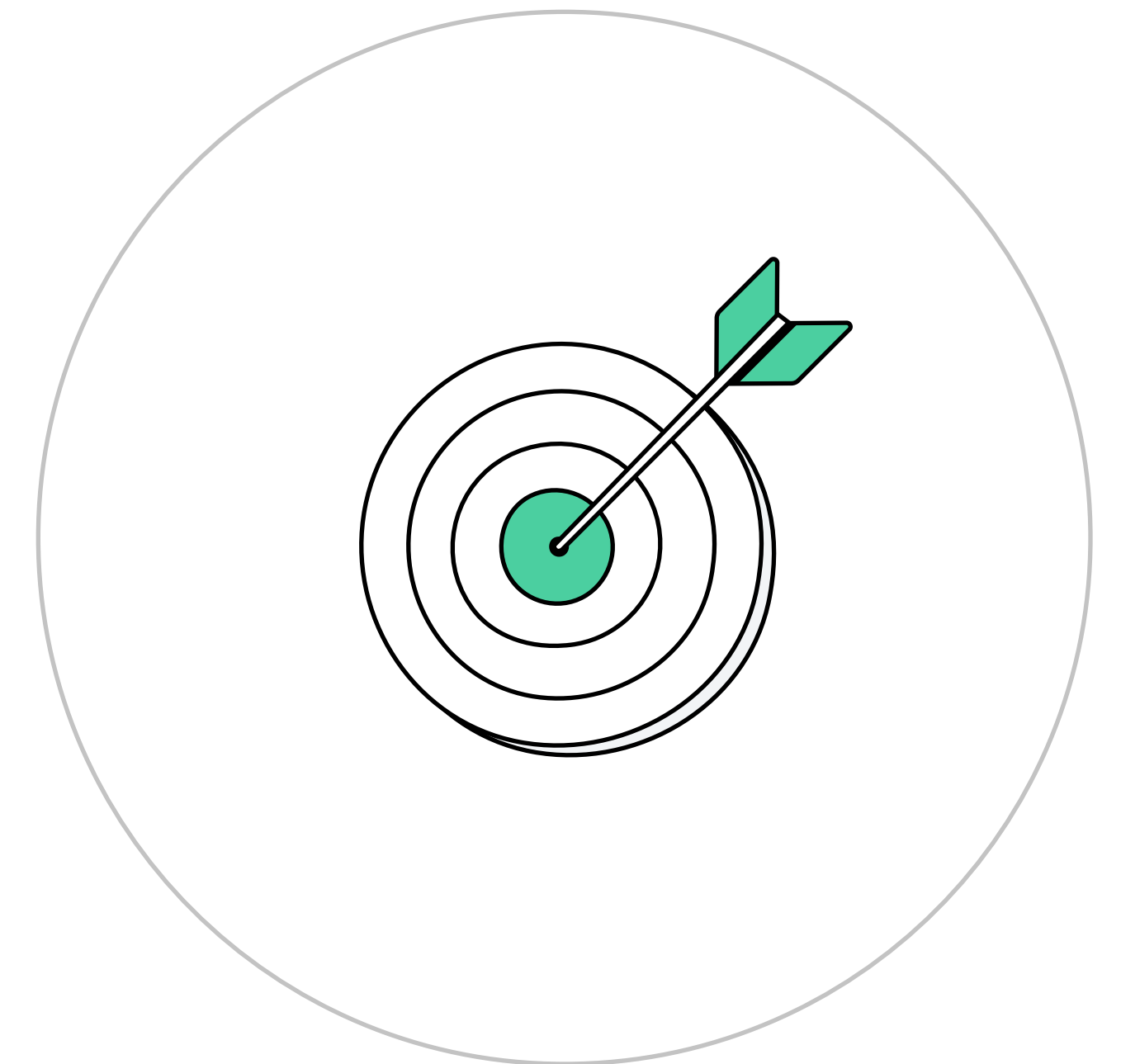


Прочие типы данных



Цели видео

- Познакомиться с интерфейсами, функциями, указателями и рунами в языке Go
- Понять основные принципы работы с ними



Прочие типы данных

К ним относятся:

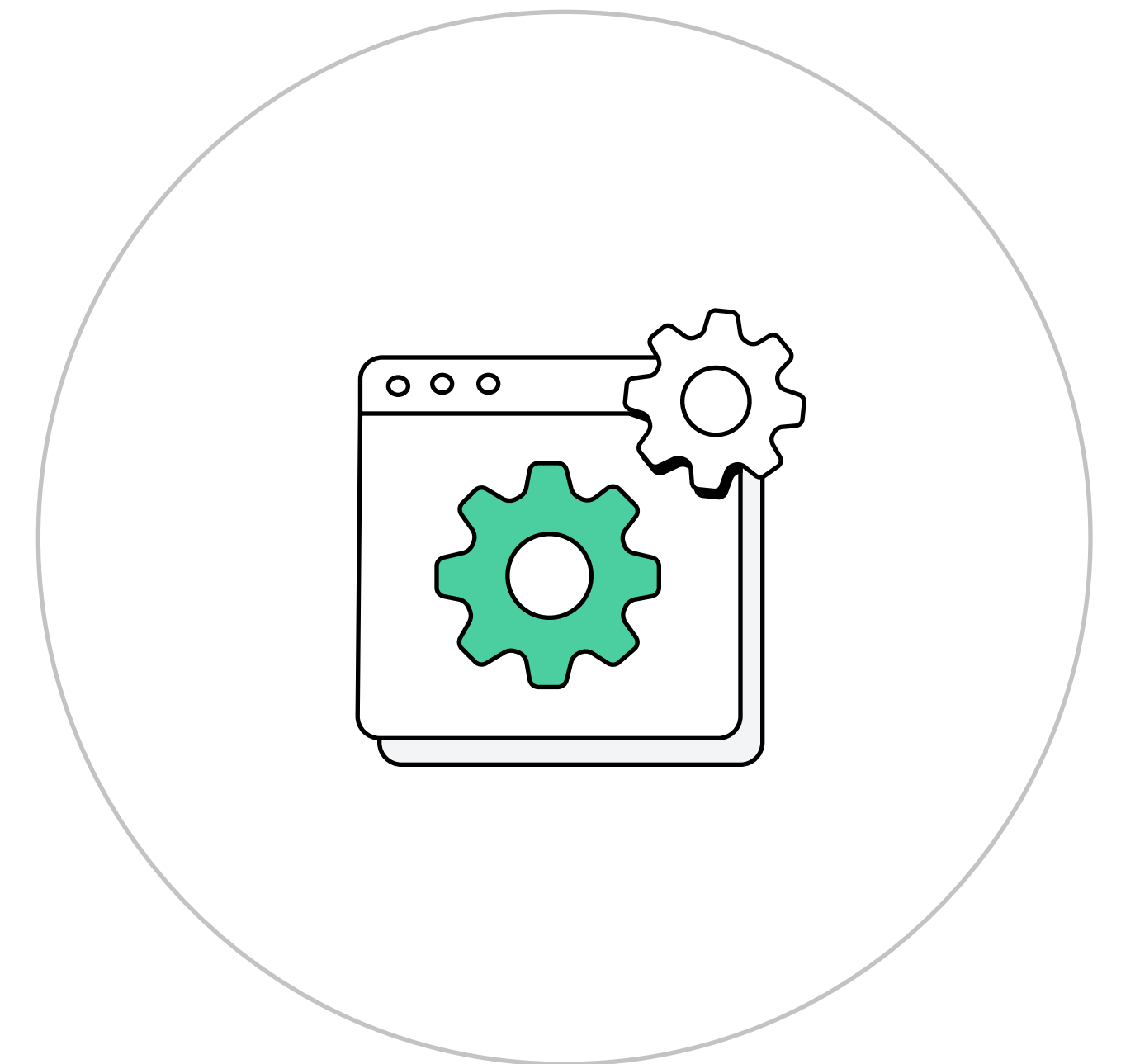
- Указатель — pointer
- Функция — function
- Интерфейс — interface
- Руна — rune



Указатель

Указатель (Pointer) — это переменная, которая хранит адрес другой переменной. Она как адрес дома — говорит вам, где найти что-то.

Указатели позволяют работать напрямую с исходными данными и нужны для эффективной работы с большими структурами данных и изменения значений внутри функций



Указатель

Примеры

```
var x int = 10  
var p *int // p – указатель на целое число  
p = &x // p теперь хранит адрес переменной x
```

- Символ * перед типом данных — объявление указателя
- & — оператор «адрес», возвращает адрес переменной

Разыменование указателя

Чтобы получить значение, которое хранится по адресу в указателе, нужно применить оператор ***** к имени указателя. Этот процесс называется **разыменованием**

```
var x int = 10  
var p *int = &x  
fmt.Println(*p) // Выведет: 10
```

***p** означает «значение, на которое указывает p»

Разыменование указателя

Мы можем изменить значение переменной, на которую указывает указатель:

```
*p = 20
```

```
fmt.Println(x) // Выведет: 20
```


Указатель. Значение nil

Nil — это специальное значение, которое показывает, что указатель не ссылается ни на какой адрес в памяти и не связан ни с одной существующей переменной.

Чтобы объявить указатель со значением nil, достаточно объявить его без инициализации:

```
var p *int // p - nil указатель на целое число
```

Указатель. Значение nil

Попытка разыменовать nil-указатель приведёт к панике (программа аварийно завершится).

Перед разыменованием указателя всегда нужно проверять, не является ли он nil:

```
var p *int
if p == nil {
    fmt.Println("указатель p равен nil")
}
```

Функция

Функция — это блок кода, предназначенный для выполнения определённой задачи.

Позволяет повторно использовать код и структурировать логику.

Объявляется с помощью ключевого слова **func**:

```
func sayHello() {  
    fmt.Println("Hello!")  
}  
  
// Главная функция. Является «точкой входа» в программу  
func main() {  
    // код тут  
}
```


Функция

```
func имя(параметры) (возвращаемые_значения) {  
    // тело функции  
    return <возвращаемые_значения>  
}
```

- func — ключевое слово
- имя — название функции
- параметры — входные данные (могут быть пустыми)
- возвращаемые_значения — одно или несколько значений, которые функция возвращает
- return — ключевое слово, прерывает выполнение функции и возвращает результат

Функция

Пример с параметрами и возвратом:

```
func add(a int, b int) int {  
    return a + b  
}  
  
func main() {  
    c := add(2, 3)  
    print(c) // 5  
}
```

- Функция add принимает два параметра типа int
- Возвращает их сумму, тоже int

Функция

Go позволяет возвращать несколько значений:

```
func divide(a, b int) (int, error) {  
    if b == 0 {  
        return 0, fmt.Errorf("деление на ноль")  
    }  
    return a / b, nil  
}
```

Эта возможность часто применяется для того, чтобы вернуть как результат выполнения функции, так и информацию об ошибке

Интерфейсы

Интерфейс — это набор методов, который описывает поведение.

Позволяет писать обобщённый код и не требует явной реализации: если тип поддерживает все методы интерфейса — он его реализует.

Делает код более универсальным и удобным для тестирования

Интерфейсы

В Go интерфейс объявляется с помощью ключевого слова **interface**:

```
type Reader interface {  
    Read(p []byte) (n int, err error)  
}
```


Пример использования интерфейса

```
type Shape interface {
    Area() float64
}

type Circle struct {
    Radius float64
}

func (c Circle) Area() float64 { // реализуем метод Area(), поэтому Circle начинает соответствовать интерфейсу
    return math.Pi * c.Radius * c.Radius
}

type Square struct {
    Side float64
}

func (s Square) Area() float64 {
    return s.Side * s.Side
}

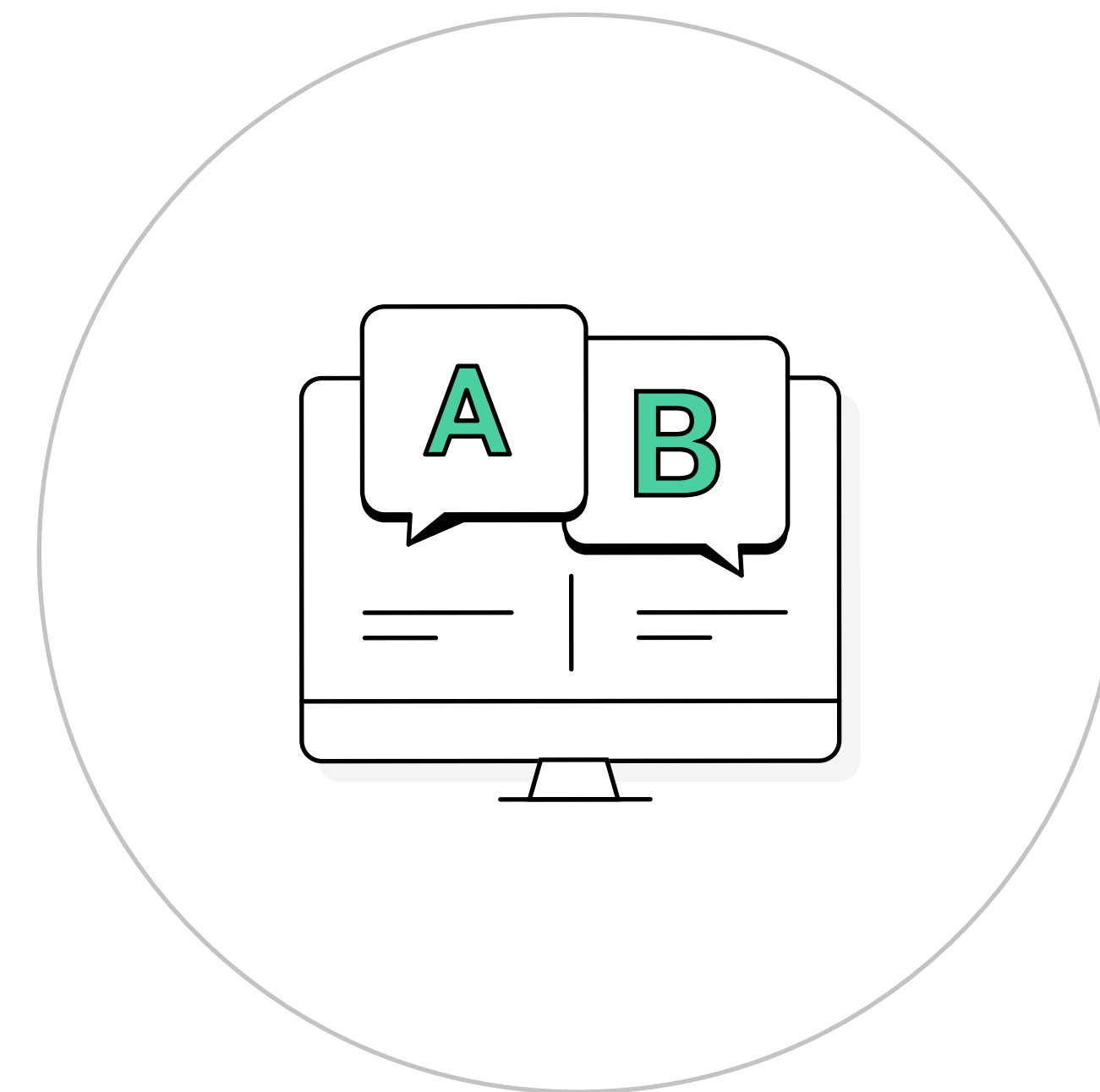
func PrintArea(s Shape) { // принимаем интерфейс Shape
    fmt.Println(s.Area())
}

func main() {
    c := Circle{ Radius: 10 }
    s := Square{ Side: 2 }
    PrintArea(c) // 314.6
    PrintArea(s) // 4
}
```

Руна

Руна (rune) - это псевдоним для `int32`. Используется для представления отдельных символов Unicode и позволяет работать с символами, выходящими за пределы ASCII.

С помощью руны можно корректно отображать символы разных языков (в том числе китайские иероглифы), эмодзи и другие специальные символы



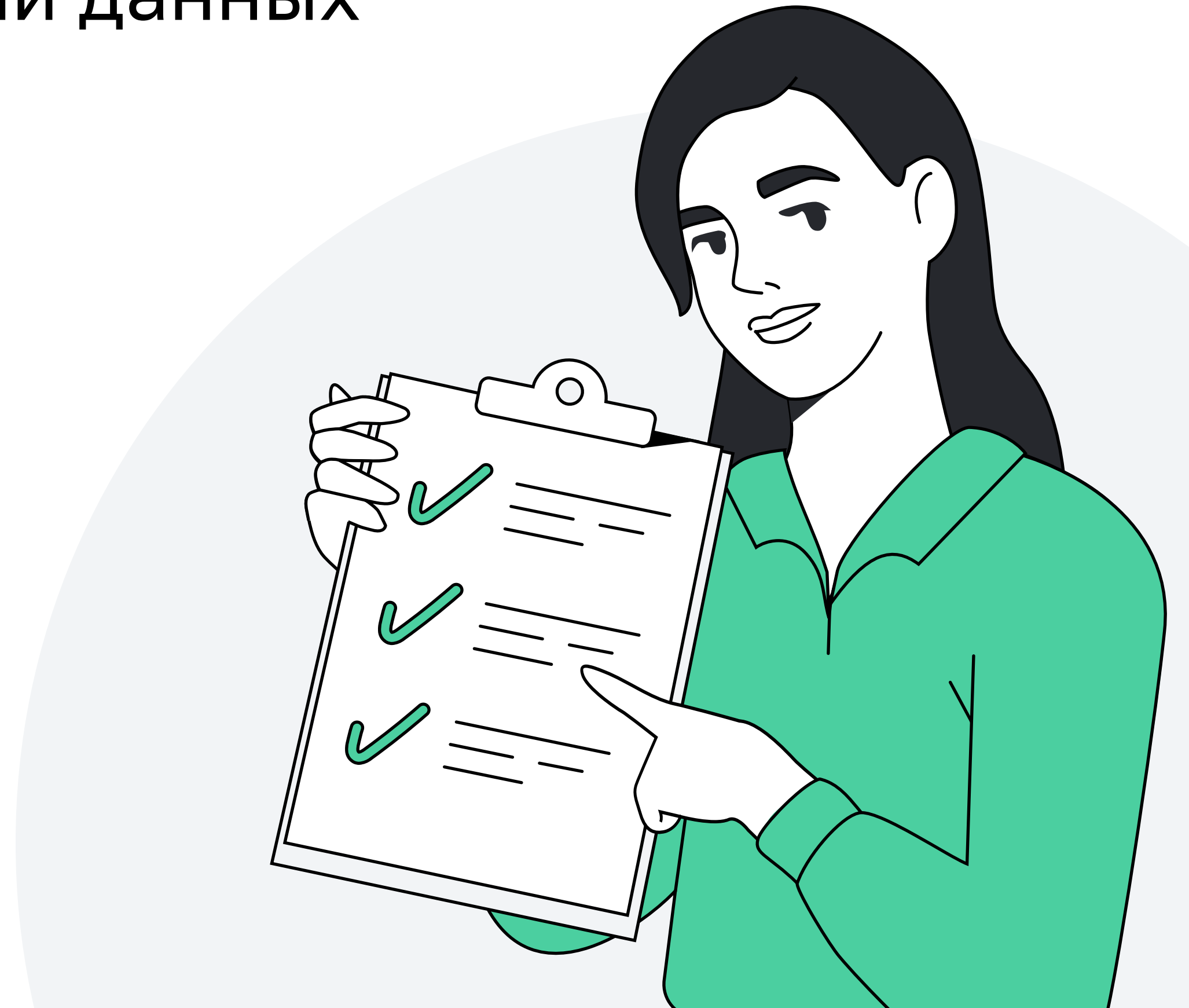
Руна

Для работы с отдельными символами необходимо преобразовать строку в срез рун посредством конструкции вида `[]rune("Строка с текстом")`

```
package main
import "fmt"
func main() {
    text := "你好，世界！Hello, World! 😊"
    fmt.Println(string(text[0])) // ä
    runes := []rune(text)
    fmt.Println(string(runes[0])) // 你
}
```


Итоги видео

- Познакомились с интерфейсами, функциями, указателями и рунами в языке Go
- Узнали, как работать с этими типами данных



Итоги занятия

- Узнали, как объявляются переменные и константы в языке Go
- Познакомились с разными типами данных в языке Go
- Поняли, как объявляются числовые, строковые и логические типы данных
- Изучили массивы, срезы, карты, структуры
- Разобрались, для чего используются интерфейсы, функции, указатели и руны



Переменные, константы, типы данных

Александр Голубь
Тимлид команды разработки

